

No Time to Evade: Context-Aware Conditional Entropy Partitioning and Model Fusion for Robust Multiprocess Malware Detection

Lara Mauri , Member, IEEE, and Ernesto Damiani , Senior Member, IEEE

Abstract—Malware detection systems are increasingly challenged by polymorphic adversarial tactics that exploit variations in the division of tasks between threads and processes. Traditional detection models trained on monolithic, single-process traces struggle to generalize across such tactics. We propose a defense framework that integrates two complementary strategies: robustness against evasion through behavioral decomposition and architectural reinforcement through model-fusion-based learning. Our method employs an entropy-based partitioning strategy to decompose system call logs while explicitly preserving temporal and semantic dependencies via a violation-aware penalty mechanism. Our partitioning scheme reflects realistic polymorphic behaviors, ensuring that functionality is preserved across partitions. The framework is further strengthened through ensemble fusion, combining multiple submodels trained under heterogeneous adversarial split configurations. We evaluate the proposed approach on the Australian Defence Force Academy Linux Dataset (ADFA-LD) benchmark dataset, its multiprocess extension, ADFA-LD-Multi-Process (ADFA-LD-MP), and on the BarongTrace dataset of real malware traces. The experimentation shows that our framework substantially improves detection performance, particularly in high-split regimes where conventional models degrade. Partition-aware training, coupled with model fusion, yields improved F1-scores and enhanced generalization across diverse adversarial conditions.

Impact Statement—Modern malware evades detection by distributing its behavior across multiple processes and execution contexts. This work introduces a scalable defense framework designed to address such evasive behaviors under realistic adversarial conditions. By combining structurally diverse models with different training strategies, our approach strengthens the detection system’s overall ability to counter evasive,

multithreaded malware. We prioritize model fusion and training diversity, reducing deployment overhead while boosting robustness. Our method seamlessly integrates with existing deep learning pipelines, making it well suited for embedded security systems.

Index Terms—Adversarial attacks, behavioral detection, ensemble learning, model fusion, polymorphic malware, robust training.

I. INTRODUCTION

MODERN malware employs sophisticated evasion techniques to bypass security controls. These techniques include *code polymorphism*, where the malware continuously alters its code structure to avoid signature-based detection; *sandbox evasion*, which allows malware to detect and avoid execution in controlled environments used for analysis; and *encrypted command-and-control* (C2) communications, which obscure malicious traffic from network monitoring tools. Additionally, “living-off-the-land” tactics leverage legitimate tools such as the victim operating system’s syscall library, making detection even more challenging [1]. Malware detectors based on machine learning (ML) face significant challenges when malware uses *behavioral splitting* (i.e., variations in the division of tasks between threads) to disguise itself. We formulate the following research questions.

- 1) [RQ1:] How can single-process malware traces be partitioned to mimic multiprocess evasion tactics while preserving semantic dependencies as well as the underlying malicious intent?
- 2) [RQ2:] How does behavioral splitting affect the performance of detection systems across different model architectures?
- 3) [RQ3:] To what extent can partition-aware training preserve detection under uncertain adversarial settings?
- 4) [RQ4:] Can model fusion compensate for the weaknesses of individual detection models, and what is the impact of ensemble size and training diversity?

Based on these research questions, we design a robust defense framework that fuses models whose training sets satisfy the semantic constraints derived from system call dependencies. Our novel contributions are summarized as follows.

- 1) We demonstrate the effectiveness of malware’s multiprocess evasion tactics in deceiving monolithic ML

Received 17 July 2025; revised 28 December 2025 and 12 January 2026; accepted 24 March 2026. Date of publication 30 March 2026; date of current version 31 August 2026. This work has been partly supported in part by the MUSA—Multilayered Urban Sustainability Action—project through the European Union—NextGenerationEU, via the National Recovery and Resilience Plan (NRRP) Mission 4 Component 2 Investment Line 1.5: Strengthening of research structures and creation of R&D “innovation ecosystems,” set up of “territorial leaders in R&D” under Grant CUP G43C22001370007 and Grant ECS00000037; and in part by the SOV-EDGE-HUB funded by Università degli Studi di Milano, Italy—PSR 2021/2022—GSA—Linea 6 under Grant CUP G45F21003110005. This article was recommended for publication by Associate Editor Chao Shen upon evaluation of the reviewers’ comments. (Corresponding author: Lara Mauri.)

The authors are with the Computer Science Department, Università degli Studi di Milano, 20133 Milan, Italy (e-mail: lara.mauri@unimi.it; ernesto.damiani@unimi.it).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TAI.2026.3678598>, provided by the authors.

Digital Object Identifier 10.1109/TAI.2026.3678598

models, leading to misclassification and drastic effects on performance.

- 2) We present a novel, entropy-based partitioning strategy that generates multiprocess instances from single-process malware traces, mirroring real-world polymorphic behaviors.
- 3) We preserve the temporal and semantic dependencies within malware system call subsequences by implementing a penalty mechanism that checks for rule-based violations, ensuring that partitioning retains malicious intent.
- 4) We propose a fusion-based framework that relies on architecture hardening and robust training to enhance the detectors' resilience against polymorphic attacks. We experimentally validate its effectiveness across different adversarial configuration scenarios.
- 5) We demonstrate the potential of model fusion combined with partitioning-aware training to compensate for model-specific architectural limitations, particularly enhancing performance across specific metrics.

The remainder of the article is organized as follows. Section II presents a literature review, focusing on hybrid and fusion-based approaches, as well as adversarial training (AT) techniques. Section III introduces some foundational concepts underlying our study, including the key principles supporting the proposed defense framework and a description of the threat model. Section IV presents our method, outlining the complementary strategies of polymorphic-aware robust training and model fusion. Section V presents the experimental evaluation of our method, assessing its effectiveness using the Australian Defence Force Academy Linux Dataset (ADFA-LD) benchmark dataset, as well as the ADFA-LD-Multi-Process (ADFA-LD-MP) dataset, a modified multiprocess version of ADFA-LD generated through our context-aware partitioning strategy. We also assess our method's robustness on *BarongTrace* [2], a reference Linux malware dataset collected via dynamic analysis in Cuckoo Sandbox and instrumented with Sysmon-for-Linux. Section VI concludes the article, suggesting directions for future research.

II. LITERATURE REVIEW

Behavioral malware features include sequences of API calls and graph representations of event-argument relationships. Most early detection methods were unsupervised. For instance, Kaspersky's approach [3] compresses behavior logs into bipartite graphs, extracts subgraphs as patterns, and uses neural networks to score suspicion levels, making the models quite resistant to evasion tactics. Supervised ML models (e.g., random forests [4]) map these features to labels (malware/benign). Deep learning (DL) models excel at handling high-dimensional data in single-model and ensemble learning settings [5]. Since our approach combines: 1) ensemble learning; and 2) adversarial-aware training via behavioral decomposition, we focus on related work belonging to these two research lines (Table I).

Recent studies [25] have demonstrated the effectiveness of fusion-based techniques based on ensemble learning, deep feature fusion, or hierarchical meta-modeling. This convergence reflects a growing recognition that single-model or uni-modal

approaches fall short in capturing the multifaceted nature of malware behavior, especially as adversaries employ evasion tactics. Classical ensemble learning techniques have primarily focused on static analysis features. For example, Damaševičius et al. [6] propose a heterogeneous ensemble combining fully connected and convolutional neural network (CNN) architectures. The final classification stage is carried out by multiple meta-learners, including K-nearest neighbors and support vector machines (SVMs), all trained on Windows Portable Executable headers [26]. Similarly, Azeez et al. [7] implemented a stacking approach to aggregate predictions through a meta-learner. These approaches reflect a broader trend toward leveraging diverse algorithms to achieve higher robustness and accuracy against malware variants. Alamro et al. [8] extended this principle to Android malware detection by employing an optimized voting ensemble of classifiers tuned with hunter-prey optimization (HPO). Their work illustrates the crucial role of hyperparameter optimization in enhancing robustness when dealing with complex malware variants. Roy et al. [9] proposed a two-layer hybrid classification framework designed specifically for malware detection, where the first layer employs three traditional classifiers (random forest, extremely randomized trees, and XGBoost), and the second layer integrates a dense NN as the meta-learner.

Building on this line of work, other studies have emphasized the importance of feature engineering. Hossain et al. [10] proposed a hybrid method integrating correlation and principal component analysis (PCA) to select the most relevant features while mitigating dimensionality issues. The resulting feature set is then used with ensemble classifiers. Zhu et al. [11] showcased the efficacy of multilevel feature correlation capture in their MEFDroid framework, where DL feature extractors are fused through innovative ensemble algorithms (ESAES, EDAES, and EDAFS) to progressively improve detection outcomes. Wang et al. [12] proposed a multistage detector that combines seven feature selection algorithms to extract permissions, API calls, and opcodes from Android apps. These features are used to train a stacking ensemble model with five base classifiers and logistic regression as the meta-classifier. Khan et al. [13] proposed a hybrid framework coupling ensemble learning with a novel CNN architecture (SB-BR-STM) to capture both global and local malicious patterns through multipath convolutions and region-boundary operations. The extracted features are classified using an ensemble of SVM, MLP, and AdaBoost. Trizna's [14] hybrid architecture adds another layer of sophistication by combining multiple DL models to analyze both contextual and behavioral features, utilizing Windows kernel emulation to capture large-scale behavioral patterns.

Novel fusion strategies have also been investigated. Johny et al. [15] introduced a DL fusion algorithm that combines multiple visual representations of malware executables. Their approach employs CNNs trained independently on each feature type and explores various fusion operators to integrate the three modalities effectively. Thakur et al. [16] combined CNN and long short-term memory (LSTM) networks to analyze binaries converted into grayscale images, extracting spatial features with CNNs and temporal dependencies with LSTMs. The extracted

TABLE I
SUMMARY OF RELATED WORKS

Hybrid and Fusion-Based Approaches			
Study	Data Type	Target Model	Fusion Architecture
Damaševičius et al. [6]	Windows PE headers (static)	FCNN + 1D CNN + meta-learners	Heterogeneous meta-learning via stacking
Azeez et al. [7]	Windows binaries (static)	NB, DT, RF, GB, AdaBoost	Classical stacking ensemble with base-learner comparison
Alamro et al. [8]	Android apps (static)	LS-SVM, KELM, RRVFLN	Optimization-driven weighted voting ensemble (HPO-tuned)
Roy et al. [9]	Obfuscated binaries	RF, ET, XGBoost + Dense NN	Hierarchical two-stage meta-learning with dense NN fusion
Hossain et al. [10]	Android apps	Various ensemble classifiers	Dimensionality-aware ensemble fusion with hybrid selectors
Zhu et al. [11]	Android apps	ESAEs, EDAEs, EDAFS (DL models)	Progressive deep ensemble with multi-scale correlation fusion
Wang et al. [12]	Android apps (static + dynamic)	5 base classifiers + Logistic Regression	Multi-stage stacked generalization with diverse feature selectors
Khan et al. [13]	Android apps (hybrid)	SB-BR-STM CNN + SVM, MLP, AdaBoost	Deep hybrid ensemble via multi-path convolutional fusion
Trizna [14]	Windows kernel behavior traces	Multiple DL models + meta-model	Behavioral-level deep ensemble with latent representation integration
Johny et al. [15]	Visual representations (images)	CNNs per modality	Late multimodal fusion using visual embeddings
Thakur et al. [16]	Grayscale binaries	CNN + LSTM + traditional classifiers	Spatio-temporal feature fusion with soft-voting aggregation
Ka et al. [17]	Static + dynamic + network traces	CNN ensembles per modality	Explainable multimodal ensemble with SHAP-aided interpretation
Adversarial Training (AT)-Like Approaches			
Study	Training Methodology	Model Type	Dataset Domain
Wang et al. [18]	GAN-based AT	2-layer architecture	Android malware detection using system call visualizations (Drebin dataset)
Alahmed et al. [19]	GAN-based AT	Random forest	Enterprise network intrusion detection using CSE-CICIDS2018 dataset
Wang et al. [20]	AT with CW attacks	LSTM-based DL model	Windows-based ransomware classification using dynamic behavioral traces from sandbox analysis
Li et al. [21]	Adversarial evaluation + training	Multiple ML models	Industrial IoT (IIoT) security monitoring with X-IIoTID dataset
Debicha et al. [22]	FGSM, BIM, PGD + AT	Deep Neural Network	Deep learning-based intrusion detection in traditional network environments
De Gaspari et al. [23]	Splitting of malware behavior	Multi-tier random forest	Behavioral ransomware detection on Windows systems using decomposed malicious processes
Mauri et al. [24]	Partitioning-aware training (Bayesian splitting)	Ensemble of models	Behavioral malware classification based on system call dependency modeling

features are used to train multiple classifiers whose outputs are combined through a voting mechanism. Other studies [17] integrated static code analysis, dynamic behavioral traces, and network traffic data using CNN ensembles tailored to each modality.

A. AT

AT emerged in response to the growing threat of adversarial attacks [27], [28] that fool detectors by feeding them inputs crafted with imperceptible perturbations. AT exposes ML-based detectors to evasive behavior already during the learning phase, improving their robustness when facing adversarially manipulated inputs at inference time. A notable contribution in this area is the AdvAndMal framework proposed by Wang et al. [18], which introduces an AT architecture specifically designed for Android malware detection and classification. AdvAndMal has a two-layer structure: a conditional generative adversarial network (GAN) to generate realistic adversarial malware samples and a classification layer that operates on RGB image representations of system call sequences. Alahmed et al. [19] applied a similar GAN-based AT strategy that was combined with a random forest classifier, while Wang et al. [20] proposed a DL-based approach using an LSTM classifier enhanced with GAN-generated adversarial examples to improve robustness against ransomware threats. Li et al. [21] proposed the EIFDAA framework, which combines adversarial evaluation and AT to assess and enhance the resilience of ML-based IDSs against a wide range of adversarial attacks. Debicha et al. [22] studied the effect of adversarial attacks on DL-based IDSs, relying on three adversarial generation techniques, namely fast gradient sign method (FGSM), basic iterative method (BIM), and projected gradient descent (PGD). Their findings highlight a tradeoff between robustness and generalization performance. AT enhances the model's robustness to adversarial perturbations but leads to a reduction in detection accuracy on nonadversarial inputs.

An emerging line of research counters structural evasion techniques that decompose malware into tasks to evade detection at runtime. A notable example is the study by De Gaspari et al. [23], which describes an evasion strategy that distributes malicious activity across multiple lightweight processes. Leveraging a multitier random forest model built on the ShieldFS

architecture, the authors train the model on partitioned ransomware samples and assess its performance under varying levels of functional splitting. Mauri et al. [24] also proposed a partitioning-aware training strategy, employing an ensemble-based approach, in which each submodel is trained on a different partition of the dataset. Their partitions are derived using a *Bayesian* splitting technique that accounts for dependencies between system calls, enabling the ensemble to better capture evasive behavior.

III. PRELIMINARIES

In this section, we introduce some foundational concepts that support our defense framework and describe the underlying threat model.

A. Semantic Dependencies Within Malware Behavior

In the context of behavioral malware analysis, *semantic dependency* refers to domain-specific knowledge about the relationships between execution events (e.g., system calls) that must be preserved to maintain the program's intended behavior. Semantic dependencies capture the logical structure underlying the system call flow, ensuring that the observed sequence reflects meaningful execution, whether in benign software or malware. Semantic constraints primarily focus on conditional dependencies, which extend beyond strict sequential ordering. That is, a system call b is not necessarily required to immediately follow a paired system call a ; rather, when a conditional system call is executed, the corresponding conditioned system call is expected to occur at some later point in the execution trace. For instance, if a program reads data from a file (`read`), it is expected to write data (`write`) at some point afterward, though not necessarily immediately after the `read`. Similarly, if memory is allocated using `malloc` (`alloc`), it is expected that the corresponding `free` call will appear later to release the memory, even if there are other operations in between.

To model the behavior of a program at the system call level, we define a *behavioral dependency graph* $G = (V, E)$, where V is a set of nodes representing system calls observed during execution, and $E \subseteq V \times V$ is a set of directed edges capturing the temporal and functional dependencies between calls [29]. Each node $v_i \in V$ can be defined as a tuple $v_i = (s_i, t_i, a_i)$,

where s_i is the system call identifier (e.g., open, read, and write), t_i is the timestamp or order index, and a_i represents the associated arguments or metadata (e.g., file descriptor and process ID). An edge $(v_i, v_j) \in E$ exists if and only if $t_i < t_j$ and there exists a functional or semantic dependency such that the execution of s_j relies on the *context* established by s_i . As an illustrative example, consider the following sequence:

`open()` $\rightarrow$ `read()` $\rightarrow$ `close()` $\rightarrow$ `fork()` $\rightarrow$ `execve()`

Here, a process opens a file using `open()`, reads data from it with `read()`, and then closes the file descriptor with `close()`. Then, the process creates a child process via `fork()`, which executes a new program through `execve()`. The edge from `open()` to `read()` represents a conditional dependency, as reading requires a valid open file descriptor. Similarly, `execve()` semantically depends on `fork()`, as the new program execution occurs in the context of the newly spawned process. Even behavioral polymorphism remains inherently bound by these fundamental semantic dependencies. We explicitly account for dependency constraints to improve robustness against multiprocess behavioral splitting.

B. Entropy-Based Modeling of Behavioral Dependencies

We introduce a statistical approach for identifying the dependencies mentioned above in execution traces. Specifically, we leverage *conditional entropy* [30] to quantify the uncertainty associated with system call transitions. Consider a time-ordered sequence of system calls $\mathcal{S} = \{s_1, s_2, \dots, s_T\}$, where each call is drawn from a finite vocabulary Σ . We define two discrete random variables X and Y representing system calls at consecutive positions t and $t + 1$, respectively. The first-order conditional entropy is given by

$$H(Y | X) = - \sum_{x \in \Sigma} P(x) \sum_{y \in \Sigma} P(y | x) \log P(y | x)$$

where $P(x)$ is the probability of observing system call x , and $P(y | x)$ denotes the conditional probability of observing y immediately following x . To account for a broader temporal context, we extend this formulation to an n -gram setting. Let $w = (s_{t-n+1}, \dots, s_{t-1}) \in \Sigma^{n-1}$ denote a context window of size $n - 1$. The conditional entropy of s_t given context w is as follows:

$$H(s_t | w) = - \sum_{w \in \Sigma^{n-1}} P(w) \sum_{s \in \Sigma} P(s | w) \log P(s | w)$$

where $P(w)$ is the probability of the context window w , and $P(s | w)$ is the probability of observing s given the preceding sequence w . This captures longer range dependencies that reflect semantically meaningful behavior. Shannon's classic information theory interprets entropy as a measure of uncertainty in a random process [30]. Applied to malware detection, it enables us to distinguish between predictable and unpredictable regions of a system call trace. By analyzing conditional entropy across the trace, we identify behavioral boundaries and split the sequence into regions. We use entropy variations to guide a partitioning strategy that maximizes the conditional entropy between adjacent subsequences.

C. Threat Model

Let $T = \{t_1, t_2, \dots, t_N\}$ denote a set of N threads involved in malware execution. Each thread t_i generates a temporal sequence of system calls $S_{t_i} = \{s_{i1}, s_{i2}, \dots, s_{iM_i}\}$, where s_{ij} represents the j th system call invoked by thread t_i , and M_i is the length of this sequence. We consider an adversary aiming to evade behavioral detection by polymorphically distributing malicious activity across disjoint execution segments (threads or subprocesses). This strategy reflects real-world malware capabilities, where malicious code spawns multiple processes or threads to fragment and obfuscate its execution trace.

1) *Adversary Knowledge (Gray-Box Setting)*: We consider an adversary operating under a *gray-box* threat model, characterized by partial yet incomplete knowledge of the detection system. Specifically, the adversary is assumed to have insight into the general nature of features utilized by the classifier (such as system call sequences, semantic dependencies, and entropy-based metrics) as well as an understanding of the overall detection methodology. However, the adversary lacks access to the detector's training data, exact model architecture, and parameters, as well as the internal decision boundaries of the classifier. While heuristic approaches can be employed to approximate detection behavior [31], full replication or inversion of the detection function remains infeasible. This partial knowledge enables the adversary to strategically partition malicious behavior across multiple processes or threads in an attempt to evade detection, subject to constraints imposed by semantic dependencies and the requirement to preserve correct execution semantics.

2) *Adversary Objective*: Given a binary classifier $f: \mathcal{V} \rightarrow \{0, 1\}$ trained to detect malicious behavior from system call traces, the adversary's goal is to produce a collection of behavioral subsequences $\{S_{t_i}\}_{i=1}^k$, where each $S_{t_i} \subseteq S_t$ for some $t \in T$, such that each subsequence is classified as benign, i.e., $f(S_{t_i}) = 0$ for all $i \in \{1, \dots, k\}$. While individual threads appear benign, their combined behavior $\bigcup_{i=1}^k S_{t_i}$ retains the full malicious semantics and functionality.

3) *Adversary Constraints*: To ensure the effectiveness of evasion, the adversary's behavior must adhere to the following constraints.

- a) *Semantic dependency integrity*: To achieve its goals, the adversary must maintain fundamental semantic constraints between system calls. This limits the adversary's ability to corrupt system call sequences, e.g., by arbitrarily suppressing calls.
- b) *No structural obfuscation via reordering*: The temporal order of system calls within each partition is preserved; the adversary cannot reorder calls in a manner that violates observed semantic or functional dependencies.
- c) *Functionality preservation*: The overall malicious functionality must remain intact when the distributed behaviors of all partitions are combined, ensuring that the malware's intended objectives are achieved despite the behavioral splitting.

Experience in the field has shown that adversaries do manipulate behavior across multiple processes—to confuse detection

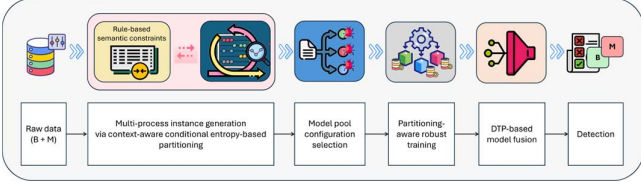


Fig. 1. Overview of the proposed model-fusion-based behavioral malware detection framework.

systems and evade analysis [32]. Our threat model formalizes realistic adversarial scenarios, in which malware authors exploit partial knowledge of detection systems to evade analysis through multiprocess behavioral splitting while remaining constrained by the semantic and functional correctness required for successful execution. A repertoire of adaptive gray-box manipulation strategies under this threat model is provided in the supplementary material.

IV. A DEFENSE FRAMEWORK

A. Methodology Overview

Our defense framework is aimed at detecting polymorphic malware that distributes its behavior across multiple threads or processes. Our approach is grounded in the observation that, despite behavioral obfuscation, the underlying semantic and temporal dependencies between system calls must remain intact for the malware to execute successfully. We exploit these invariant dependencies through a defense architecture that integrates two complementary strategies: 1) *robust training via behavioral decomposition*; and 2) *architectural reinforcement through model-fusion-based learning*. As illustrated in Fig. 1, the framework comprises the following three interdependent components.

- 1) A *splitting strategy* decomposes system call sequences into semantically meaningful partitions.
- 2) A *robust training regime* leverages these partitions to improve model generalization and robustness.
- 3) An *architectural reinforcement mechanism* integrates the outputs of multiple learning models to boost resilience.

At the core of our method lies a *context-aware, conditional entropy-based* splitting strategy, which partitions system call sequences into behaviorally coherent subsequences. Rather than relying on a monolithic model, we train a set of independent detection models on these partitions, whose outputs are subsequently combined via a model-fusion layer to enhance overall detection accuracy and robustness.

Definition (Context). In this work, *context* refers to the local behavioral environment surrounding each system call within a sequence. Formally, given a sequence of system calls $S = [s_1, s_2, \dots, s_n]$, the context $\mathcal{C}_i$ for any system call s_i is defined as the set of system calls within a fixed-size sliding window centered around s_i . For a window radius $w \in \mathbb{N}$,

$$\mathcal{C}_i = \{s_j \mid i - w \leq j \leq i + w, j \neq i, 1 \leq j \leq n\}.$$

This context captures local dependencies and patterns in the neighborhood of each system call, reflecting the behavioral environment at position i .¹

1) *Behavioral Splitting*: In the following, we describe the two splitting strategies used in our framework: a *context-aware entropy-based* method for malicious sequences and a *round-robin (RR) approach* for benign ones.

a) *Conditional entropy-based splitting (malicious sequences)*: Given a sequence of system calls S executed by a single thread, the objective is to partition each malicious sequence into k nonoverlapping subsequences $\{P_1, P_2, \dots, P_k\}$ such that each split preserves domain-specific dependencies between system calls and reflects distinct behavioral patterns.

To identify partition boundaries, we leverage *context-aware conditional entropy* as a quantitative indicator of local behavioral uncertainty. Intuitively, positions exhibiting abrupt increases in uncertainty (reflected as peaks in the entropy) are considered likely candidates for behavioral transitions.

Definition (Entropy Trace). Given a sequence of system calls $S = [s_1, s_2, \dots, s_n]$, the *entropy trace* is defined as the ordered sequence of local conditional entropy values computed for each position i , based on its surrounding behavioral context $\mathcal{C}_i$. Formally, the entropy trace is as follows:

$$\mathcal{H}(S) := [H(s_1 \mid \mathcal{C}_1), H(s_2 \mid \mathcal{C}_2), \dots, H(s_n \mid \mathcal{C}_n)]$$

where each term $H(s_i \mid \mathcal{C}_i)$ is the conditional entropy of s_i given its context $\mathcal{C}_i$, defined as $H(s_i \mid \mathcal{C}_i) = -\sum_{s \in \mathcal{V}} P(s_i = s \mid \mathcal{C}_i) \log P(s_i = s \mid \mathcal{C}_i)$. This trace captures the local behavioral uncertainty throughout the sequence, so peaks in it indicate positions where the system's behavior may transition.

To identify suitable split points, we focus on local maxima in the entropy trace. Formally, the set of candidate split points is defined as

$$\mathcal{T} = \{i \in [2, n-1] \mid H(s_i \mid \mathcal{C}_i) > H(s_{i-1} \mid \mathcal{C}_{i-1}) \text{ \& } H(s_i \mid \mathcal{C}_i) > H(s_{i+1} \mid \mathcal{C}_{i+1})\}$$

which corresponds to positions where the conditional entropy is higher than that of their immediate neighbors.

From this candidate set, we select the top- k positions with the maximum entropy values

$$\mathcal{T}_k = \arg \max_{|\mathcal{T}|=k} \sum_{i \in \mathcal{T}} H(s_i \mid \mathcal{C}_i)$$

subject to the constraint that for all $j \in \{1, \dots, k-1\}$, the minimum distance between split points satisfies $t_{j+1} - t_j \geq L_{\min}$, ensuring that each segment is sufficiently large and nontrivial.

To prevent splits from violating dependency relationships among system calls, we introduce a regularization term that balances the tradeoff between entropy maximization and semantic integrity. This ensures that the resulting partitions are

¹Efficient computation of conditional entropy can be made possible by limiting the context to a small, fixed window of preceding events. For instance, by considering a sliding window of radius $w = 1$, which accounts only for the immediately preceding system call, the context for s_i reduces to the conditional distribution $\mathcal{C}_1(s_i) := P(s_i \mid s_{i-1})$, $2 \leq i \leq n$, thereby effectively capturing local dependencies through pairs of consecutive system calls.

Algorithm 1 Context-Aware Conditional Entropy-Based Splitter.

Require: System call sequence $S = [s_1, \dots, s_N]$, split factor k , context definition $\mathcal{C}$, minimum distance $L_{\min}$
Ensure: Optimal split points $\mathcal{T}^*$ inducing k subsequences

```

1: Compute entropy trace  $\mathcal{H}(S) = [H(s_1|\mathcal{C}_1), \dots, H(s_N|\mathcal{C}_N)]$ 
2: Candidate peak set:  $\mathcal{T} \leftarrow \{i \in [2, N-1] \mid H(s_i|\mathcal{C}_i) > H(s_{i-1}|\mathcal{C}_{i-1}) \wedge H(s_i|\mathcal{C}_i) > H(s_{i+1}|\mathcal{C}_{i+1})\}$ 
3: Sort  $\mathcal{T}$  in descending order of  $H(s_i|\mathcal{C}_i)$ 
4: Initialize  $\mathcal{T}_k \leftarrow \emptyset$ 
5: for all  $i \in \mathcal{T}$  do
6:   if  $|\mathcal{T}_k| < k-1$  and  $(\mathcal{T}_k = \emptyset \text{ or } \min_{t \in \mathcal{T}_k} |i - t| \geq L_{\min})$  then
7:     Add  $i$  to  $\mathcal{T}_k$ 
8:   end if
9: end for
10: Induce  $k$  subsequences  $\{\sigma_1, \dots, \sigma_k\}$  from  $\mathcal{T}_k$ 
11: for  $j = 1$  to  $k$  do
12:   Compute  $H(\sigma_j) = \sum_{s_i \in \sigma_j} H(s_i|\mathcal{C}_i)$ 
13:   Compute  $\text{Penalty}(\sigma_j) = \left(\frac{H(\sigma_j)}{|\sigma_j|}\right) \cdot |\mathcal{U}(\sigma_j)| + V(\sigma_j)$ 
14: end for
15: Compute objective  $\mathcal{J}(\mathcal{T}_k) = \sum_{j=1}^k H(\sigma_j) - \sum_{j=1}^k \text{Penalty}(\sigma_j)$ 
16: Set  $\mathcal{T}^* \leftarrow \mathcal{T}_k$ 
17: return  $\mathcal{T}^*$ 

```

behaviorally diverse while preserving both the temporal order and functional dependencies critical to correct execution. Preserving such structural relationships is crucial as it enables the partitioned sequences to more accurately reflect the underlying intent of the malicious execution. We define the penalty function as follows.

Given a set of split points $\mathcal{T}_k$ that induces k subsequences $\{\sigma_1, \dots, \sigma_k\}$, the penalty associated with each split σ_j is defined by

$$\text{Penalty}(\sigma_j) = \left(\frac{H(\sigma_j)}{|\sigma_j|}\right) \cdot |\mathcal{U}(\sigma_j)| + V(\sigma_j)$$

where $H(\sigma_j)$ denotes the sum of conditional entropy values over the split σ_j , $|\sigma_j|$ is the length of the split, $\mathcal{U}(\sigma_j) = \{s_i \mid s_i \in \sigma_j, i \leq \lceil 0.1 \times |\sigma_j| \rceil\}$ denotes the set of unique system calls occurring within the first decile² of split σ_j , and $V(\sigma_j) \in \mathbb{N}_0$ counts the semantic violations within σ_j .

Incorporating the penalty term, the objective of the partitioning process is to maximize the overall entropy across splits while minimizing violations of conditional dependencies

$$\mathcal{J}(\mathcal{T}) = \sum_{j=1}^k H(\sigma_j) - \sum_{j=1}^k \text{Penalty}(\sigma_j).$$

The optimal set of split points is obtained as

$$\mathcal{T}^* = \arg \max_{\mathcal{T}, |\mathcal{T}|=k-1} \mathcal{J}(\mathcal{T}).$$

The complete context-aware conditional entropy-based splitting procedure is summarized in Algorithm 1.

Round-Robin Splitting (Benign Sequences): For benign sequences, we adopt a RR assignment strategy. Each system call is assigned to one of the k splits in a fixed, cyclical fashion: the i th system call is allocated to $\sigma_{(i \bmod k) + 1}$. This strategy is intentionally nonadversarial and structure-agnostic, preserving the statistical and temporal characteristics of the original trace without introducing artificial dependencies.

²The first decile of each split is analyzed to quantify early behavioral diversity, as this portion often encapsulates foundational system-level operations that establish the execution context and influence downstream control flow. Violations in the early-part of the split incur a higher penalty.

The use of RR splitting ensures that any observed robustness gains are attributable to the adversarially aware partitioning of malicious traces rather than to artifacts introduced into benign inputs. By treating benign and malicious sequences asymmetrically, we emulate realistic threat scenarios (see Section III-C), in which only malicious behaviors are subject to evasion strategies [33], while benign activity remains unaltered. This asymmetry is essential to isolate the effectiveness of our defense mechanism. We further validate this assumption through a sensitivity analysis,³ where benign traces are also partitioned using alternative control strategies.

B. Robust Training

Once each sequence has been partitioned into k parts, the resulting splits serve as training units. Let $\mathcal{D} = \{(S_i, y_i)\}_{i=1}^N$ denote the original dataset, where S_i is a system call sequence and $y_i \in \{0, 1\}$ is its label (malicious or benign). We define a selected set of split factors $\mathcal{K}_{\text{train}} \subset \mathbb{N}$, which represents the values of k used during training. For each $k \in \mathcal{K}_{\text{train}}$, we apply the splitting strategy to each sequence S_i , yielding a set of k subsequences $\{\sigma_i^{(1,k)}, \dots, \sigma_i^{(k,k)}\}$, all confined to the same dataset split. This generates a transformed training set

$$\mathcal{D}^{(k)} = \left\{ \left(\sigma_i^{(j,k)}, y_i \right) \mid 1 \leq i \leq N, 1 \leq j \leq k \right\}.$$

Each partitioned subsequence retains the label y_i of its original full trace and is treated as an independent training instance. For each $k \in \mathcal{K}_{\text{train}}$, we train a separate detection model $f_k : \Sigma^* \rightarrow [0, 1]$, resulting in a set of models: $\mathcal{F} = \{f_k \mid k \in \mathcal{K}_{\text{train}}\}$.

Compared to conventional AT (see Section II), our polymorphic-aware training avoids reliance on gradient-based perturbations, which may produce semantically invalid or unrealistic samples [34], [35]. Instead, our conditional entropy-based splitting strategy simulates adversarial conditions without explicitly crafting formal adversarial examples. This approach effectively mimics real-world polymorphic malware behavior, in which malicious functionality is fragmented and distributed across multiple processes. Such fragmentation poses challenges for traditional AT, as it requires careful design to ensure that generated samples accurately represent actual behavior [34].

C. Architecture Reinforcement via Model Fusion

The final stage of our defense framework aggregates the predictions of the submodels through a fusion mechanism, which acts as an architectural reinforcement layer. Rather than relying on a single monolithic model trained on a uniform behavioral view, the ensemble integrates multiple diverse detectors trained under distinct partitioning regimes. An effective model fusion scheme is essential for leveraging the diversity introduced by our context-aware splitting strategy and mitigating the limitations of individual submodels. We employ a combiner function that implements a *deny-takes-precedence* (DTP) decision rule, which is based on a conservative fusion principle inspired by

³Fully reported in the paper's supplementary material.

TABLE II
COMPARISON BETWEEN ADFA-LD AND ADFA-LD-MP DATASETS

	ADFA-LD	ADFA-LD-MP
Purpose	Standard HID evaluation	Adversarial polymorphic evaluation
Data Type	Raw system call traces	System call subsequences derived from original traces
Execution Model	Single-process traces	Simulated multiprocess traces
Trace Count	833 (training), 4372 (validation), 746 (attack)	Scaled proportionally according to k
Splitting Strategy	None	Malign: context-aware conditional entropy-based partitioning Benign: round-robin assignment

the *minority report* paradigm, as proposed in [24]. Given an input sequence x , the ensemble prediction $\hat{y}(x)$ is defined as

$$\hat{y}(x) = \begin{cases} 1, & \exists m \in \{1, \dots, M\} \text{ such that } f^m(x) = 1 \\ 0, & \text{otherwise} \end{cases}$$

where M is the total number of submodels in the ensemble, and $f^m(x) = 1$ indicates that submodel m classifies the sequence as malicious ($f^m(x) = 0$ as benign). The DTP rule ensures that the ensemble predicts a sequence as malicious if *any* submodel signals malicious behavior, regardless of the majority prediction.

Our use of the DTP-based combiner is rooted in its capacity to minimize false negatives. Ensuring that any indication of maliciousness, even if detected by a single model, is sufficient to trigger an alert is essential, especially in scenarios where undetected malicious behaviors are distributed sparsely across temporal windows or obfuscated within benign activity.

V. EVALUATION

This section describes the experiments we conducted to test the effectiveness and robustness of our scheme.

A. Experimental Settings

We detail the experimental settings, present the results and a critical analysis of their implications.

1) *Benchmark Datasets*: We assessed our system's performance under conventional (single-process) and polymorphic-like (multiprocess) conditions by leveraging two datasets: the well-known ADFA-LD benchmark dataset [36] and a modified version we introduced, named ADFA-LD-multi-process (ADFA-LD-MP). The original ADFA-LD dataset is a widely used benchmark for host-based intrusion detection (HID) systems that includes system call traces from both benign and malicious executions under various attack scenarios on an Ubuntu Linux distribution. It consists of 833 normal training traces, 4372 normal validation traces, and 746 attack traces. Each trace, represented as a sequence of integers, corresponds to a single-process execution and captures the system call behavior of that process. Table III details the distribution of system call traces, while Table IV presents the attack vectors used to generate the ADFA-LD malicious traces. To simulate realistic multiprocess malware behavior, we generated the ADFA-LD-MP dataset by decomposing each malicious execution trace from the original ADFA-LD into k subsequences. This decomposition is guided by our context-aware conditional entropy-based splitting algorithm, with k ranging from 2 to 40. Table II summarizes the key differences between the two datasets.

TABLE III
SYSTEM CALL TRACES DISTRIBUTION FOR ADFA-LD DATASET

	Traces	System Calls
Training Data	833	308 077
Validation Data	4372	2 122 085
Attack Data	746	317 388
Total	5951	2 747 550

TABLE IV
ATTACK VECTORS USED TO GENERATE THE ADFA-LD DATASET

Attack	Payload/Effect	Vector	Trace Count
Hydra-FTP	Password bruteforce	FTP by Hydra	162
Hydra-SSH	Password bruteforce	SSH by Hydra	176
Adduser	Add new superuser	Client side poisoned executable	91
Java-Meterpreter	Java based Meterpreter	TikiWiki vulnerability exploit	124
Meterpreter	Linux Meterpreter Payload	Client side poisoned executable	75
Webshell	C100 Webshell	PHP remote file inclusion vulnerability	118

2) *Experimental Campaign*: To evaluate the efficacy of the proposed framework, we conducted an experimental campaign investigating various aspects of the polymorphic detection pipeline under both adversarial and nonadversarial conditions. Specifically, the campaign comprises the following stages.

- 1) [C1:] We assess the performance of baseline DL models in nonadversarial conditions, where both training and testing are performed on standard single-process system call sequences. This establishes an empirical upper bound and serves as a reference for quantifying degradation under polymorphic evasion.
- 2) [C2:] We evaluate the impact of inference-time polymorphic splitting on baseline models trained on nonadversarial data, validating the gray-box threat model.
- 3) [C3:] We simulate an informed defense by training monolithic models on datasets partitioned according to the attacker's known split configuration. This setup enables us to evaluate whether alignment with behavioral decomposition allows models to recover their baseline performance in controlled scenarios.
- 4) [C4:] We analyze the performance of our fusion-based detection framework, which aggregates predictions from submodels trained on traces partitioned under diverse splitting granularity. By systematically varying the number and heterogeneity of training configurations and submodels, we quantify the resilience of the fusion architecture under different polymorphic scenarios.

Each experiment was repeated ten times. The execution platform used for the experimentation is a 64-bit Microsoft Windows 11 Pro operating system, running on an Intel(R) Core(TM) Ultra 7 155H processor (1.40 GHz) with

32 GB of RAM. The experimentation was conducted using the Python programming language (v3.12.7) and the following libraries: TensorFlow (v2.18.0), Pandas (v2.2.2), NumPy (v1.26.4), Scikit-learn (v1.5.1), Matplotlib (v3.9.2), and Seaborn (v0.13.2).

3) *Model Fusion Setup*: In our defense framework, we employ a fusion strategy based on an ensemble of behavioral submodels, each trained using our entropy-based sequence partitioning. As discussed in Section IV-B, we denote by $\mathcal{K}_{\text{train}} \subset \mathbb{N}$ the set of split factors used to generate the partitioned training sets $\mathcal{D}_{\text{tr}}^p(k)$.⁴ For each $k \in \mathcal{K}_{\text{train}}$, we train a model $f_k : \Sigma^* \rightarrow \{0, 1\}$, resulting in a pool of candidate detectors $\mathcal{F}_{\text{pool}} = \{f_k \mid k \in \mathcal{K}_{\text{train}}\}$.

Each submodel $f_k \in \mathcal{F}_{\text{pool}}$ is implemented as either a LSTM network or a temporal convolutional network (TCN), capturing complementary temporal representations of system call sequences. These architectures are selected for their ability to model sequential dependencies in noisy or polymorphic behavioral traces [37], [38], [39].

To construct the detection ensemble, we select a subset of M models from $\mathcal{F}_{\text{pool}}$, defining the active ensemble as $\mathcal{F} = \{f_{k_1}, f_{k_2}, \dots, f_{k_M}\} \subseteq \mathcal{F}_{\text{pool}}$, where each f_{k_i} is associated with a split degree $k_i \in \mathcal{K}_{\text{train}}$ and a model type (either LSTM or TCN). The configuration of $\mathcal{F}$ allows for diverse ensemble designs, focusing on fine-grained (k small), coarse-grained (k large), or mixed partitioning levels for increased robustness.

We consider ensembles of varying sizes, with $M \in \{3, 7, 10\}$, allowing us to see the impact of diverse behavioral traces on different ensemble configurations. The training set choices for $\mathcal{F}_{\text{pool}}$ include the following: 1) small k values to simulate fine-grained splitting, typical of malware exhibiting minimal process-level dispersion; 2) large k values (up to 40 splits per sequence) to simulate coarse-grained behaviors associated with more obfuscated process activities; and 3) heterogeneous combinations of k values to capture multiscale behavioral variability present in complex multiprocess malware. These training scenarios are evaluated on polymorphic test sets $\mathcal{D}_{\text{ts}}^p(k)$, where the split factor k is varied to simulate different degrees of obfuscation and process-level dispersion.

4) *Evaluation Metrics*: We employ both standard classification metrics (Accuracy = $\text{TP} + \text{TN} / \text{TP} + \text{TN} + \text{FP} + \text{FN}$, Precision = $\text{TP} / \text{TP} + \text{FP}$, Recall = $\text{TP} / \text{TP} + \text{FN}$, and F1-score = $2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$) and targeted evaluation criteria. Specifically, we report the classical metrics as primary performance indicators, each computed under different polymorphic attack conditions, and then apply three complementary evaluation criteria tailored to the multiprocess threat model, all grounded in the experimental campaign outlined in Section V-A2. First, we define *polymorphic degradation* ($\Delta_p(k)$) as

$$\Delta_p(k) = \text{Perf}(f^s, \mathcal{D}_{\text{ts}}^p(k)) - \text{Perf}(f^s, \mathcal{D}_{\text{ts}}^s) \quad (1)$$

⁴The superscript p in $\mathcal{D}^p(k)$ denotes datasets (either training (tr) or test (ts)) consisting of polymorphic malware traces, characterized by multiprocess behavior, whereas the superscript s refers to datasets containing single-process traces.

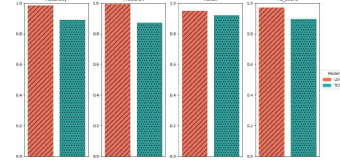


Fig. 2. Performance comparison of baseline DL models.

which measures the loss incurred by a model f^s trained on single-process data $\mathcal{D}_{\text{tr}}^s$ when evaluated on partitioned sequences $\mathcal{D}_{\text{ts}}^p(k)$. Second, we define *training soundness* as the condition

$$\text{Perf}(f^p, \mathcal{D}_{\text{ts}}^p(k)) > \text{Perf}(f^s, \mathcal{D}_{\text{ts}}^p(k)) \quad (2)$$

which reflects detection improvement from polymorphic-aware training. Third, we define *ensemble robustness* as the ability of a fusion model $\mathcal{E}$ trained under polymorphic conditions to maintain or improve detection performance compared to both the monolithic baseline and its individual submodels when evaluated on multiprocess data, formally

$$\text{Perf}(\mathcal{C}(\mathcal{E}), \mathcal{D}_{\text{ts}}^p(k)) > \max \left\{ \text{Perf}(f^s, \mathcal{D}_{\text{ts}}^p(k)), \max_m \text{Perf}(f_m^p, \mathcal{D}_{\text{ts}}^p(k)) \right\}. \quad (3)$$

5) *Rule-Based Constraints*: In addition to preserving temporal consistency in syscall sequences, our defense strategy leverages a set of rule-based constraints that encode semantic dependencies between system calls. These constraints reflect domain knowledge about typical operating system behavior and serve to enhance the behavioral coherence of extracted subsequences [40]. Specifically, we define a collection of directed syscall dependency rules of the form (s_i, s_j) , which assert that the invocation of syscall s_j is expected to follow s_i within a semantically valid execution flow. For instance, $\text{open}() \rightarrow \text{read}()$ means that $\text{read}()$ is expected only after a file has been successfully opened via $\text{open}()$.

These dependencies are encoded as constraints via the penalty function and integrated into the malware partitioning process, as described in Section IV-A1. During partitioning, deviations from these semantic rules are penalized, thereby encouraging the alignment of subsequences with plausible functional boundaries. Table V summarizes a nonexhaustive set of system call dependency pairs considered during this stage, derived from the x86-64 system call list as documented in the ADFA-LD dataset GitHub repository [41].

B. Results and Analysis

Fig. 2 shows the performance of the LSTM and TCN baseline monolithic models, where both training and testing were conducted using nonpolymorphic system call sequences.

The LSTM model achieved an accuracy of 98.35%, with a precision of 99.26%, a recall of 94.96%, and an F1-score of 97.06%, indicating highly reliable detection with minimal false positives. The TCN model, while slightly less effective, still achieved an accuracy of 89.04%, a precision of 87.09%, a recall of 92.00%, and an F1-score of 89.48%. These results establish

TABLE V
RULE-BASED SEMANTIC CONSTRAINTS FOR LINUX SYSTEM CALL DEPENDENCY PAIRS, DERIVED FROM THE x86-64 SYSCALL LIST [41]

System Call Dependency	Category	Constraint Description
File and Directory Management		
openat()/open() → read() openat()/open() → write() openat()/open() → close() openat()/open() → unlinkat()/unlink() creat() → write() mkdirat()/mkdir() → rmdir() chdir() → openat()/open() fstat()/stat() → openat()/open() lseek() → read()/write()	File Access File Access File Lifecycle File Deletion File Creation Directory Lifecycle Directory Navigation File Status File Positioning	File must be opened before it can be read. File must be opened before it can be written. File descriptor must be closed after use. File must be opened before deletion. Create file before writing. Directory must be created before removal. Change working directory before opening files. File status checked before opening. File offset set before reading or writing.
Memory Mapping		
mmap()/mmap2 → munmap() mmap()/mmap2 → mprotect() mmap()/mmap2 → msync() brk() → sbrk()	Memory Mapping Memory Protection Memory Synchronization Heap Management	Must unmap after mapping. Change protection after mapping. Synchronize mapped memory to disk. Adjust program break before allocating memory.
Process Management		
fork()/clone() → execve() fork()/clone() → exit() waitid()/wait4() → exit() execve() → exit() getpid() → fork()/clone()	Process Lifecycle Process Lifecycle Synchronization Process Lifecycle Process Identification	Child executes new binary after fork. Child exits after execution. Parent waits for child before exiting. Process exits after execve failure or termination. Get PID before forking new process.
File Descriptors and I/O		
pipe()/pipe2 → read() pipe()/pipe2 → write() dup()/dup3() → close() fcntl() → ioctl() openat()/open() → fcntl()	Pipe I/O Pipe I/O FD Duplication FD Control FD Control	Can read only after pipe creation. Can write only after pipe creation. Duplicate before closing. Descriptor manipulated before IOCTL. File descriptor opened before control operations.
Socket and Networking		
socket() → bind() socket() → connect() bind() → listen() listen() → accept()/accept4() connect() → sendto()/sendmsg() connect() → recvfrom()/recvmsg() sendto()/sendmsg() → shutdown() recvfrom()/recvmsg() → shutdown()	Socket Binding Socket Init Server Setup Connection Handling Client I/O Client I/O Socket Shutdown Socket Shutdown	Bind only after socket creation. Connect only after socket creation. Listen after binding. Accept after listening. Send only after connection. Receive only after connection. Shutdown after sending data. Shutdown after receiving data.
Synchronization and Miscellaneous		
futex() → futex() timerfd_create() → timerfd_settime() epoll_create() → epoll_wait() signalfd() → read()	Synchronization Timer Usage Event Polling Signal Handling	Futex used in coordinated locking patterns. Timer must be created before setting time. Polling after epoll instance creation. Read signal after signalfd setup.

an upper bound benchmark under conditions where the models have full access to the original behavioral traces and can learn temporal correlations in the absence of adversarial polymorphic strategies. This serves as a reference point for understanding the impact of behavioral splitting that we introduced in the subsequent experiments.

1) *Threat Model Validity*: As formalized in our threat model (Section III-C), the adversary seeks to evade detection by decomposing the full malicious behavior S_t into a set of disjoint subsequences $\{S_t^i\}_{i=1}^k$ intended to appear benign, such that $f(S_t^i) = 0 \forall i \in \{1, \dots, k\}$. To validate our threat model, we first analyze the impact of polymorphic splitting on performance metrics across a varying number of splits and then quantify the degradation relative to baseline monolithic models through a Δ analysis comparing multiprocess (attacked) to single-process (baseline) scenarios. As shown in Fig. 3, both baseline monolithic models exhibit a monotonic degradation in their ability to detect malicious activity as the number of polymorphic splits k increases. In terms of accuracy, the LSTM model shows a decline from 97.08% at $k = 2$ to 95.66% at $k = 5$. Even a small degree of polymorphic behavior substantially impairs detection capabilities [24]. A similar trend is observed for recall, which starts at 97.87% at $k = 2$, then drops to 87.07%, 92.12% and 91.44% at $k = 3$, $k = 4$, and $k = 5$, respectively. In contrast, precision remains high and consistent, ranging from 98.87% to 100% across all values of k . This stability indicates that the model’s ability to correctly identify positive instances is unaffected by the increased number of splits, aligning with our goal to make the model susceptible primarily to multiprocess instances that emulate polymorphic malware behavior. The F1-score, which balances recall and

precision, decreases from 98.36% at $k = 2$ to 91.44% at $k = 5$, reinforcing the overall impact of recall loss. Turning to the TCN model, it achieves an accuracy of 86.08% at $k = 2$ and drops sharply to 75.20% at $k = 4$, demonstrating a more pronounced reduction in performance with the introduction of the 4-split-based behavior. The most significant drop occurs between $k = 2$ and $k = 3$, with accuracy falling by approximately 11%. Unlike the LSTM model, the TCN model is more sensitive in terms of recall. It achieves an average recall of 72.60% for $k \leq 5$, compared to 92.13% for the LSTM. Regarding overall accuracy, the TCN achieves a mean of 80.50%, whereas the LSTM consistently outperforms it with an average accuracy of 95.55% in the same range. This discrepancy can be attributed to fundamental design differences: LSTMs capture long-term dependencies essential for modeling polymorphism due to sequence splitting, while TCNs prioritize short-range patterns, making them less effective under behavioral fragmentation. Performance trends for larger splits ($10 \leq k \leq 20$) further confirm this behavior. The LSTM model experiences a mean drop of 10.88% in accuracy, 22.44% in recall, and 9.26% in F1-score relative to $k = 5$. TCN exhibits significantly worse degradation: mean losses of 22.77% (accuracy), 53.86% (recall), and 35.33% (F1-score). For $k > 25$, both models show near-total performance collapse, reaching approximately 50% accuracy at $k = 40$, which is equivalent to random guessing. Although LSTM maintains perfect precision, it suffers critically in recall and F1-score, struggling to identify positive instances despite low false positive rates, whereas TCN breaks down across all metrics, including precision.

A further confirmation of the effectiveness of the evasion strategy emerges when comparing the above results to the

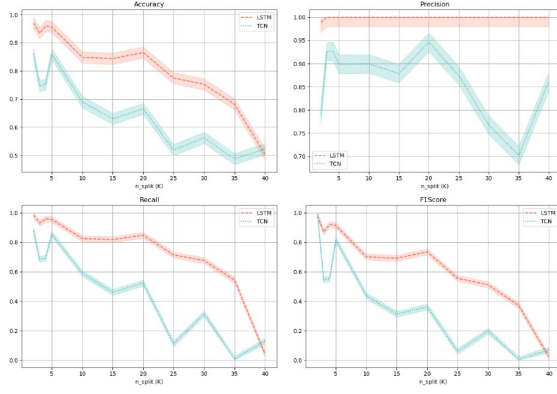


Fig. 3. Performance impact of multiprocessing splitting across different k values on monolithic models.

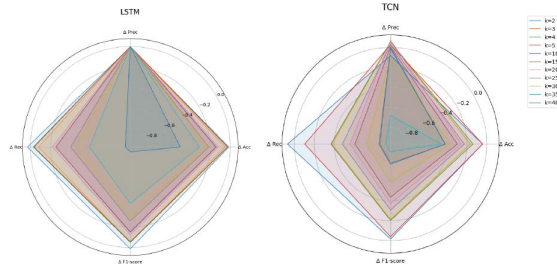


Fig. 4. Relative performance degradation (Δ) of LSTM and TCN models under multiprocessing polymorphic splitting attacks compared to monolithic baselines. Negative values indicate declines in accuracy, precision, recall, and F1-score as the number of splits k increases.

baseline scenario, where detection models operate on monolithic system call traces. Fig. 4 illustrates the relative performance degradation (Δ), computed according to (1) as the difference between each evaluation metric under attack and its corresponding baseline value, across varying values of k .

Adversarial and Defensive Implications: From an adversarial perspective, our results confirm the empirical observation [32] that behavioral decomposition is an effective strategy for evading detection. As the number of segments k increases, the performance of classifiers degrades markedly. However, this strategy also imposes constraints on the adversary, who must balance the degree of decomposition with the need to preserve the malware functional integrity, timing, and inter-process dependencies. Excessive decomposition risks breaking these dependencies, making the attack ineffective.

From a defensive standpoint, our findings underscore the inherent limitations of current monolithic sequence-based models, which are typically trained on single-process data and fail to capture behavior across multiple concurrent processes. Addressing this gap necessitates a paradigm shift in detection architectures toward models capable of operating over distributed behavioral contexts and leveraging shared dependency structures, such as the robust fusion-based approach proposed in our framework.

2) Soundness Under Behavioral Splitting: We evaluate soundness by assessing whether polymorphic-aware training

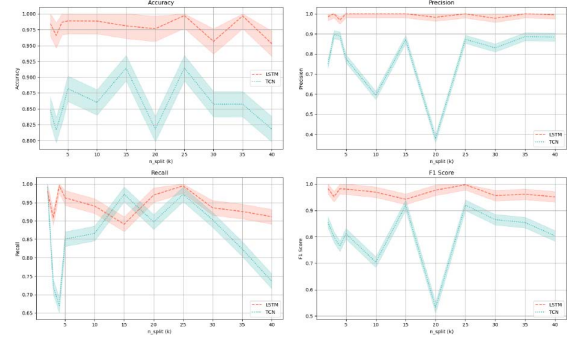


Fig. 5. Performance under informed defense across different process splits k on monolithic models.

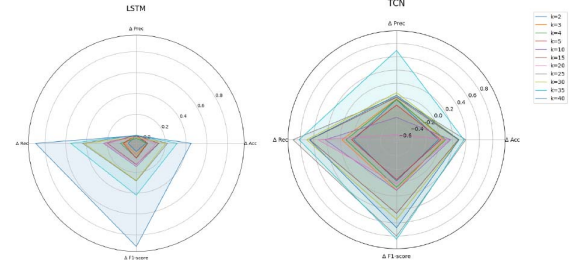


Fig. 6. Relative performance gain (Δ) of LSTM and TCN models under informed training with multiprocessing polymorphic splitting attacks, compared to monolithic baseline models.

enhances detection performance in scenarios where the defender has prior knowledge of the adversary's splitting strategy and trains the (monolithic) model on data partitioned using the same multisplit configuration. In this way, we simulate an informed defense in which the training data reflects the structure of polymorphic attacks in terms of the number of behavioral splits.

As shown in Fig. 5, polymorphic-aware training yields notable improvements in detection performance. For instance, LSTM accuracy and F1-score remain consistently above 95.2% and 94.2% for all k values, respectively. LSTM precision consistently exceeds 96.9%, while recall exhibits some variability—most pronounced at moderate partition splits—but remains consistently above 89.1% across all polymorphic splits. In contrast, the TCN model demonstrates greater performance variability. While its accuracy remains consistently above 81.6%, precision and recall exhibit more substantial fluctuations, ranging approximately from 59.3% to 88.9% and 67.1% to 99.3%, respectively.

In Fig. 6, we report the obtained percentage gain in detection performance compared to the baseline threat scenario.

Both models occasionally surpass their respective baselines obtained under nonpolymorphic training and testing. Notably, in some configurations, the LSTM model achieves higher accuracy and F1-scores than in the baseline scenario. For instance, at $k = 25$, LSTM attains an accuracy of 99.73% and an F1-score of 99.74%, compared to the baseline values of 98.35% and 97.06%, respectively. The TCN model, while more sensitive to variations in partitioning, also shows performance gains in

specific settings. For instance, at $k = 15$ and $k = 25$, TCN precision improves from the baseline value of 87.09% to 87.31% and 87.32%, respectively; recall increases from 92.00% to 97.23% in both cases, resulting in F1-scores of up to 92.01%. These results confirm that, given exact knowledge of the polymorphic splitting strategy, both LSTM and TCN models can effectively leverage polymorphic-aware training to counteract the obfuscation effects of behavioral decomposition, thereby improving detection performance relative to the monolithic baseline.

Limitations: The above observation is contingent on the assumption that the defender has prior knowledge of the attacker’s splitting strategy. In practical deployments, such exact knowledge is rarely available, and polymorphic behavior may vary over time. As a result, training detection models exclusively on datasets partitioned using a fixed splitting factor may yield suboptimal generalization and performance degradation when the actual polymorphic behavior deviates from that assumed during training. Moreover, defending against multiple partitioning strategies necessitates training and maintaining separate models for each splitting configuration, imposing significant computational overhead. These intrinsic limitations motivate the development of adaptive defense strategies that integrate knowledge across multiple partitioning configurations, thereby potentially enhancing robustness under adversarial uncertainty.

3) *Penalty Ablation and Context-Size Sensitivity:* Our splitting strategy relies on a composite penalty function that integrates three complementary terms. Candidate cut points are first identified based on salient variations in the entropy trace and are then refined using a penalty that combines normalized entropy, early-segment diversity $|\mathcal{U}(\sigma)|$, and rule-based semantic violations $V(\sigma)$. To quantify the contribution of each component and assess the sensitivity of the entropy term to context size, we conduct an ablation study by selectively removing individual terms and measuring the resulting change in F1-score relative to the full penalty.

We consider the following variants: a) *No penalty* (entropy peaks only); b) removal of the early-diversity term $|\mathcal{U}(\sigma)|$; and c) removal of the semantic-violation term $V(\sigma)$. All models are trained under the informed-defense setting with $k_{\text{train}} = 10$, which serves as a reference partitioning regime that preserves sufficient contextual information while inducing controlled exposure to split variability. Evaluation is performed under increasingly fine-grained splitting regimes $k_{\text{test}} \in \{10, 25, 40\}$. Results are reported as ΔF1 , where negative values indicate degradation relative to the full objective.

As shown in Fig. 7, removing the penalty results in the largest performance degradation across all splitting regimes. At $k_{\text{test}} = 10$, the drop in F1-score is limited (approximately $\Delta\text{F1} \approx -0.03$ for LSTM and -0.04 for TCN), but increases steadily with split granularity, reaching roughly -0.13 (LSTM) and -0.15 (TCN) at $k_{\text{test}} = 40$. This pattern indicates that entropy-only selection tends to over-emphasize noisy transition points, with errors amplified as subsequences become shorter and increasingly fragmented. Removing the early-diversity term $|\mathcal{U}(\sigma)|$ yields smaller but consistent degradations. For both architectures, ΔF1 remains close to zero at $k_{\text{test}} = 10$, while increasing to approximately -0.04 (LSTM) and -0.05 (TCN)

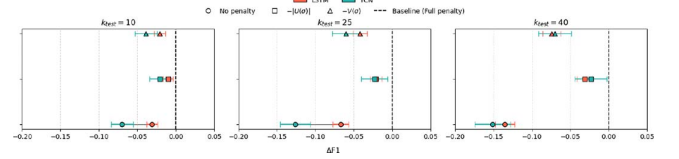


Fig. 7. Penalty ablation expressed as ΔF1 relative to the full objective across increasing test-time splitting regimes k_{test} .

TABLE VI
SEMANTIC VIOLATIONS AND
CLASSIFICATION ERROR RATES
UNDER DIFFERENT SPLITTING
STRATEGIES, AVERAGED
ACROSS TEST-TIME SPLITTING
REGIMES $k_{\text{test}} \in \{10, 25, 40\}$

Strategy	Viol./Seq.	Err. (%)
No partitioning	0.6	6.2
Entropy-only	3.8	14.9
– $ \mathcal{U}(\sigma) $	2.1	10.7
– $V(\sigma)$	2.9	12.8
Full penalty	1.2	7.4

at $k_{\text{test}} = 40$. This suggests that early-segment diversity contributes primarily at higher split granularity, acting as a regularizing mechanism that limits early heterogeneity and reduces variance across subsequences. Ablation of the $V(\sigma)$ term produces a more pronounced decline. Already at $k_{\text{test}} = 25$, performance drops by approximately $\Delta\text{F1} \approx -0.04$ for LSTM and -0.05 for TCN, increasing further to around -0.07 and -0.09 , respectively, at $k_{\text{test}} = 40$, suggesting that explicit dependency constraints contribute to the preservation of execution-level coherence under fine-grained splitting regimes. The near-parallel degradation patterns observed for LSTM and TCN suggest that the penalty operates primarily at the level of input structure. TCN models exhibit larger absolute deltas across all ablations, reflecting a higher sensitivity to semantically inconsistent local patterns.

Table VI reports semantic violation statistics and corresponding classification error rates for the full splitting and its ablation variants, averaged across test-time splitting regimes. The *no partitioning* baseline exhibits a small but nonzero number of violations, reflecting noise in real execution traces and the intentionally incomplete coverage of the dependency rules rather than splitting-induced inconsistencies. Entropy-only splitting leads to an increase in both violation counts and error rates, although the magnitude of this increase varies across configurations. Incorporating the full penalty reduces violations relative to entropy-only splitting but does not eliminate them entirely, resulting in violation degrees that remain slightly above the no-partitioning reference. Ablation variants exhibit intermediate and nonuniform behavior, indicating that individual penalty components contribute differently across settings. Overall, while the relationship is not strictly monotonic, higher semantic violation rates tend to be associated with increased misclassification errors.

With respect to the context definition used in conditional entropy estimation, we evaluate sensitivity by varying the n -gram window size $n \in \{2, 3, 4, 5\}$ while keeping the splitting objective and all other hyperparameters fixed. As in the ablation

TABLE VII
SENSITIVITY OF CONDITIONAL
ENTROPY-BASED SPLITTING TO THE
 n -GRAM CONTEXT SIZE

n -Gram Window	LSTM (Avg. F1)	TCN (Avg. F1)
$n = 2$	0.707	0.681
$n = 3$	0.748	0.726
$n = 4$	0.742	0.729
$n = 5$	0.721	0.703

study, all configurations are trained under the informed-defense regime with $k_{\text{train}} = 10$ and evaluated across test-time splitting regimes $k_{\text{test}} \in \{10, 25, 40\}$. For each context size, we report the average F1 across test-time splitting regimes (Table VII). Smaller contexts ($n = 2$) are associated with lower average F1-scores across both architectures, indicating that very short contexts provide insufficient conditioning for reliable entropy estimation. Conversely, larger contexts ($n = 5$) also lead to a decline in performance, suggesting that overly long contexts attenuate informative variations in the entropy trace.

As shown in Table VII, intermediate context sizes yield the strongest overall performance, although the optimal n differs slightly across architectures. For LSTM models, $n = 3$ achieves the highest average F1-score, while $n = 4$ remains close but lower. In contrast, TCN models attain their best average performance at $n = 4$, with $n = 3$ yielding comparable but slightly reduced results. Despite these differences, both architectures exhibit their strongest performance at moderate context sizes, indicating that entropy estimation over short-to-medium contexts yields a stable operating regime for entropy-based splitting.

4) *Comparison of Splitting Strategies:* We investigate the impact of the partitioning mechanism by comparing our context-aware conditional entropy-based splitting with the Bayesian dependency-aware approach proposed in [24]. The Bayesian splitting procedure is adapted to operate under the same experimental constraints as our entropy-based method by enforcing the generation of exactly k contiguous subsequences per trace for a given split factor k , relying on the same set of system call dependency pairs, and explicitly matching the ensemble size to k . This design choice differs from the robust fusion setting analyzed later and is adopted here to isolate the effect of the splitting strategy itself. Given its stochastic nature, Bayesian splitting is executed multiple times for each k , with results averaged across ten runs. Performance is reported in terms of $\Delta F1$ relative to the no-splitting monolithic baseline.

As shown in Fig. 8, both methods yield positive $\Delta F1$ values for moderate split factors, indicating that controlled behavioral decomposition can improve detection performance. As the split granularity increases, however, Bayesian splitting exhibits higher variance and earlier performance degradation, particularly for TCN-based models. In contrast, the entropy-based strategy maintains more stable $\Delta F1$ values across both architectures. These trends reflect distinct design objectives. Bayesian splitting emphasizes probabilistic dependency inference, which introduces flexibility but also increases sensitivity to stochastic effects and fine-grained partitioning. By contrast,

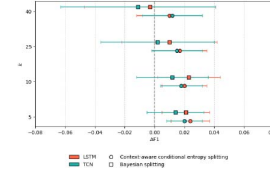


Fig. 8. Comparison between our context-aware conditional entropy-based and Bayesian splitting strategies: ensemble-level $\Delta F1$ relative to the no-splitting baseline across split factors k .

TABLE VIII
SPLITTING GRANULARITY CONFIGURATIONS

M	Training k -Values	Granularity Behavior
3	{2, 3, 4}	Fine-grained
3	{5, 10, 15}	Mid-grained
3	{20, 25, 30}	Coarse-grained
3	{5, 25, 40}	Heterogeneous
7	[2 — 20]	Fine-to-mid
7	[10 — 40]	Mid-to-coarse
10	[2 — 35]	Broad heterogeneous

entropy-based splitting is explicitly optimized for stability under controlled partitioning, resulting in more consistent behavior as k increases.

5) *Robust Model Fusion-Based Detection:* In this section, we present the results of our defense approach. Our analysis centers on how behavioral splitting granularity affects model training and how ensemble size and diversity impact detection performance, with comparisons to monolithic models trained on single-process sequences. We restrict our evaluation to polymorphic test sets with splitting degrees ranging from 10 to 40 to focus on worst case scenarios characterized by moderate to high behavioral dispersion. As we observe in the first experiment, this range aligns with conditions, under which baseline monolithic models exhibit substantial performance degradation. Here, we focus on accuracy and F1-score due to their operational relevance. Table VIII summarizes the ensemble configurations used.

a) *Effect of splitting granularity:* First, we investigate the influence of the splitting degree k on the detection efficacy of ensembles comprising $M = 3$ submodels. The area charts in Fig. 9 show the performance deltas in accuracy and F1-score across the selected test set range for the different granularity configurations. Regardless of the training data's splitting granularity, all configurations show consistent improvements over the monolithic baseline. In particular, under the fine-grained regime, the LSTM achieves a peak improvement of approximately 39% in accuracy and 80% in F1-score at $k = 40$, while the TCN attains a comparable accuracy gain of around 35%, but exhibits a more modest F1-score improvement of approximately 22%. Moving from the mid-grained to the heterogeneous configuration, both ensembles demonstrate an enhanced capacity to detect behaviors with higher degrees of splitting compared to the fine-grained training configuration, reaching, for example, an accuracy delta of 49% and a $\Delta F1$ -score of 95% at $k = 40$. In contrast, the coarse-grained configuration shows limited improvements at lower splitting degrees and even negative F1-score deltas initially, suggesting difficulties in generalizing from overly split behavioral sequences. Although

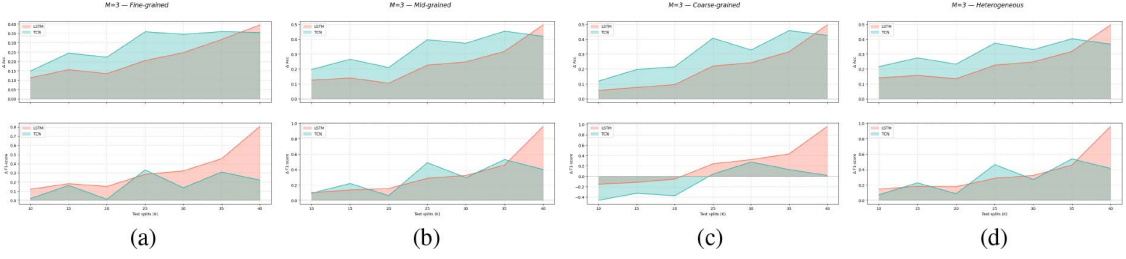


Fig. 9. Δ comparison across different splitting granularity configurations for $M = 3$.

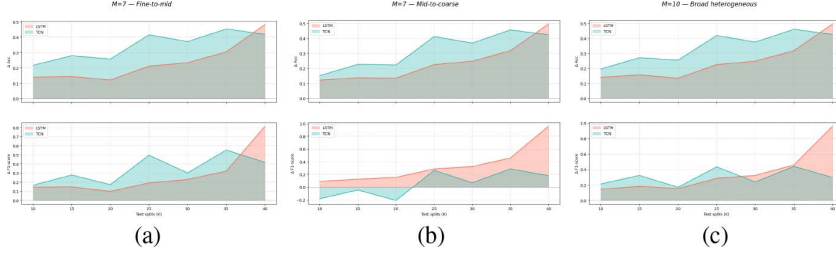


Fig. 10. Δ comparison across different splitting granularity configurations for $M = 7$ and $M = 10$.

the impact is somewhat constrained by the limited number of submodels in the ensemble, the heterogeneous configuration proves to be the most robust, effectively restoring detection capability against polymorphic evasion across both low and high splitting degrees by leveraging the strengths of diverse training granularity.

b) False positive impact of DTP fusion: We compare DTP fusion with majority voting to assess false positive behavior under the same test-time split conditions considered above. The results indicate that DTP consistently preserves higher recall across split factors, at the cost of an absolute increase in false positive rate of approximately 1.5%–3.0% points relative to majority voting. Importantly, this increase remains bounded and stable across fragmentation levels, whereas majority voting exhibits a more pronounced degradation in recall as splitting becomes increasingly aggressive. These findings suggest that the choice of fusion rule provides a practical mechanism for controlling the false positive–false negative balance at deployment time without requiring retraining of the underlying submodels.

c) Impact of ensemble size and diversity: Looking at Fig. 10, we observe that increasing the ensemble size to $M = 7$ yields measurable benefits under the fine-to-mid training configuration, particularly at lower splitting levels in the polymorphic test set. When compared to the fine-grained configuration with $M = 3$, the LSTM-based ensemble shows improved performance at the extremes of the splitting spectrum (i.e., $k = 10$ and $k = 40$), both in terms of accuracy and F1-score. For instance, it achieves a relative accuracy gain of +24.5% and +21.8% at $k = 10$ and $k = 40$, respectively, along with a moderate improvement at $k = 25$ (+3.3%), while exhibiting slightly diminished performance in the intermediate range. As for the TCN-based ensemble, we observe consistent improvements across all k values in both accuracy and F1-score, with

notable gains at lower splitting degrees. For example, Δ accuracy increases from 14.8% to 21.7% at $k = 10$, while Δ F1-score improves substantially from 1.1% to 17.3%. Similarly, when compared to the most effective $M = 3$ configuration (i.e., the heterogeneous setup), the $M = 7$ ensemble shows mixed results. LSTM-based models perform slightly worse in most k values, meaning that a broader training distribution does not necessarily yield stronger generalization when model diversity is not increased enough. In contrast, the TCN-based models consistently benefit from the larger ensemble, achieving higher F1-score deltas across all splitting degrees, with particularly large relative improvements at low k values. The ensemble with mid-to-coarse training configuration exhibits a performance trend broadly aligned with that of the coarse-grained configuration with $M = 3$ but achieves systematically higher accuracy and F1-score, particularly at lower k values. Nonetheless, its overall effectiveness remains inferior to the $M = 3$ heterogeneous ensemble, which demonstrates superior generalization due to its greater training diversity. The above results indicate a tradeoff between ensemble size and training diversity: larger ensembles with homogeneous training granularity yield performance gains primarily within specific splitting regimes, whereas heterogeneous ensembles, leveraging diverse training granularity, achieve more robust detection across a wider spectrum of polymorphic splitting degrees. Last, with $M = 10$, we observe that the broad heterogeneous ensemble retains the same accuracy and F1-score performance as the $M = 3$ heterogeneous case for the LSTM-based models, while the TCN-based ensemble shows marked improvements in F1-score at lower k values (e.g., from 7.1% to 21.3% at $k = 10$).

d) Discussion: Integrating multiple submodels trained on partitioned data enhances detection robustness against multi-process adversarial strategies. Ensembles with diverse splitting

granularity consistently outperform those trained on homogeneous partitions, underscoring the critical role of behavioral variation in enabling robust generalization. While increasing ensemble size alone yields marginal improvements, particularly in the absence of training diversity, the combination of model fusion with heterogeneous training configurations effectively compensates for inherent architectural limitations. This effect is especially pronounced in TCN models, which exhibit marked sensitivity to behavioral decomposition.

C. Robustness Across Datasets

We extend our robustness analysis by evaluating the proposed partitioning-aware framework on *BarongTrace* [2], a real-world Linux malware dataset collected via dynamic analysis in Cuckoo Sandbox and instrumented with Sysmon-for-Linux. *BarongTrace* provides process-level telemetry events generated during the execution of 22 784 Linux ELF binaries (10 414 malicious and 12 370 benign), yielding a balanced corpus of short and multithreaded traces. In contrast to ADFA-LD, whose long system-call sequences exhibit substantial redundancy, *BarongTrace* traces are typically only tens of events long and inherently encode concurrent execution. These properties impose strict semantic constraints on admissible partitioning. In particular, meaningful split factors are limited to $k \in \{1, 2, 5, 10\}$, as larger values would trivially collapse execution context. As TCNs exhibit stabler training dynamics on short and fragmented traces, we focus on them for a reliable assessment of partitioning effects. Following the same protocol adopted for ADFA-LD, we train single- k TCN models at $k_{\text{train}} \in \{2, 5, 10\}$ and evaluate them under mismatched test-time partitioning. Robustness is expressed in terms of deviation from the monolithic baseline, enabling a direct comparison with ADFA-LD-MP despite differences in event vocabularies and trace lengths.

Table IX reports the $\Delta F1$ values for all single- k TCN models together with the DTP-based ensemble. Training at low fragmentation ($k_{\text{train}} = 2$) yields negative $\Delta F1$ values for $k_{\text{test}} = 2$ and $k_{\text{test}} = 5$. This limited exposure to partitioning is insufficient to induce invariance even to moderate test-time segmentation. Only at $k_{\text{test}} = 10$ the model exhibits a slight performance recovery, suggesting that very coarse partitioning attenuates spurious local correlations rather than fostering genuine structural robustness. Training at intermediate fragmentation ($k_{\text{train}} = 5$) produces the most balanced behavior among single models. Although performance remains negative at $k_{\text{test}} = 2$, the model achieves its highest positive $\Delta F1$ at the matched condition $k_{\text{test}} = 5$ and remains near-neutral at $k_{\text{test}} = 10$. This pattern mirrors, in a weaker form, the stabilization effect observed on ADFA-LD-MP, where moderate partitioning encourages the model to internalize structural features that generalize across nearby partitioning scales. Training at the highest partitioning degree $k_{\text{train}} = 10$ is consistently detrimental, yielding negative $\Delta F1$ across all test conditions. Unlike ADFA-LD-MP, where redundancy mitigates aggressive splitting, *BarongTrace*'s short traces amplify information loss at high k . The DTP ensemble follows an upper envelope behavior, matching the strongest

TABLE IX
ROBUSTNESS ON *BARONGTRACE*: $\Delta F1$ RELATIVE TO THE
MONOLITHIC BASELINE FOR SINGLE- k TCN MODELS
AND THE DTP-BASED ENSEMBLE ($M = 3$)

	$k_{\text{test}} = 2$	$k_{\text{test}} = 5$	$k_{\text{test}} = 10$
$k_{\text{train}} = 2$	-0.04 ± 0.031	-0.05 ± 0.028	$+0.03 \pm 0.034$
$k_{\text{train}} = 5$	-0.05 ± 0.029	$+0.06 \pm 0.033$	-0.01 ± 0.027
$k_{\text{train}} = 10$	-0.05 ± 0.021	-0.03 ± 0.024	-0.01 ± 0.022
DTP Ensemble	-0.05 ± 0.019	$+0.05 \pm 0.021$	$+0.02 \pm 0.018$

single- k detector at each k_{test} while avoiding the worst degradations. It remains slightly negative at $k_{\text{test}} = 2$, where all components are impaired, and becomes positive at $k_{\text{test}} \in \{5, 10\}$ when at least one model captures useful structural invariance.

Overall, these findings show that the robustness patterns observed on ADFA-LD extend to a substantially different setting, albeit under stricter structural constraints. While short Sysmon traces are more sensitive to aggressive partitioning, controlled exposure to partition variability and multiscale fusion remain effective in stabilizing detection performance.

D. Empirical Insights

Our research questions aimed to understand how the interplay between partitioning-aware training, model architecture, and ensemble-based defenses influences the robustness of malware detection systems against polymorphic evasion. Five main findings emerge, as follows.

[F1:] *Single-model architectures are structurally vulnerable to behavioral decomposition, with robustness varying according to their sequence modeling capacity.* Increasing the degree of process splitting significantly impairs detection performance. Architectures equipped with advanced temporal modeling mechanisms, such as LSTMs, are slightly more robust, as they partially preserve inter-subsequence context. In contrast, models such as TCNs, which rely on fixed temporal kernels, suffer pronounced performance degradation across all metrics.

[F2:] *Partitioning-aware training can restore detection; however, when used alone in single-model architectures, it demands significant computational resources and is limited by the need for precise prior knowledge.* Informed training strategies can effectively recover or improve detection performance by addressing the challenges posed by multiprocess evasion tactics. When applied in isolation within monolithic models, this approach faces limitations. First, it demands considerable computational effort, as it requires training on multiple partitioning configurations and handling diverse behavioral splits. Second, the approach heavily relies on accurate prior knowledge of the adversary's splitting strategy, making it less effective in real-world dynamic or unpredictable adversarial environments.

[F3:] *Context-aware conditional entropy-based training, in conjunction with DTP-based model fusion, enhances robustness to polymorphic attacks.* Ensembles trained using our context-aware conditional entropy-based partitioning strategy, combined with DTP-based model fusion, significantly outperform monolithic models. This advantage is particularly evident in high- k regimes.

[F4:] *Training diversity outweighs the benefits of simply enlarging ensemble size.* Incorporating diverse training split configurations has a greater impact than merely expanding the number of submodels. Specifically, while larger homogeneous ensembles provide only incremental gains, smaller ensembles that integrate varying partition granularity demonstrate superior performance across different adversarial conditions.

[F5:] *Model fusion compensates for architectural deficiencies, particularly in F1-score performance.* Our results underscore the compensatory property of the multimodel approach in addressing inherent architectural deficiencies. Models such as TCNs, which are prone to significant performance degradation under high degrees of behavioral splitting, exhibit limitations in both precision and recall. However, when integrated into an ensemble trained on diverse splitting configurations, these weaknesses are mitigated, thus balancing the precision-recall tradeoff. This results in significant F1-score improvements.

VI. CONCLUSION AND OUTLOOK

Our results demonstrate that ensemble models allow for finer-grained behavioral capture and mitigate the dilution of malicious signals that often occurs in whole-trace models. However, as adversaries evolve, future malware is expected to adopt even more sophisticated evasion strategies, including cross-process coordination, intermittent execution, distributed payload delivery across networked systems, and adaptive behavior randomly triggered by environmental cues [42]. To counter these strategies, our future research will explore temporal ensemble learning, graph-based behavioral correlation across processes, and multi-host trace aggregation to develop explainable AI techniques suitable for keeping malware detection capable of ensuring trust in operational cybersecurity environments.

REFERENCES

- [1] K. Scarfone and P. Mell, "Guide to intrusion detection and prevention systems (IDPS)," Gaithersburg, MD, USA: National Institute of Standards and Technology (NIST) Special Publication, vol. 800, 2007, p. 94.
- [2] B. A. Pratomo, S. A. Kosim, H. Studiawan, and A. O. Prabowo, "BarongTrace: A malware event log dataset for Linux," in *Proc. Int. Conf. Adv. Inf. Netw. Appl.*, Cham, Switzerland: Springer Nature, 2024, pp. 48–60.
- [3] A. Chistyakov, A. Shevelev, A. Romanenko, and E. Lobacheva, "Monotonic models for real-time dynamic malware detection," in *Proc. 6th Int. Conf. Learn. Represent., ICLR, Workshop Track Proc.*, 2018.
- [4] M. Morcos, H. Al Hamadi, E. Damiani, S. Nandyala, B. and McGillion, A., "Surrogate-based technique for android malware detectors' explainability," in *Proc. 18th Int. Conf. Wireless Mobile Comput., Netw. Commun. (WiMob)*, Piscataway, NJ, USA: IEEE, 2022, pp. 112–117.
- [5] J. Ferdous, R. Islam, A. Mahboubi, and M. Z. Islam, "A review of state-of-the-art malware attack trends and defense mechanisms," *IEEE Access*, vol. 11, pp. 121118–121141, 2023.
- [6] R. Damasevicius, A. Venckauskas, J. Toldinas, and S. Grigaliunas, "Ensemble-based classification using neural networks and machine learning models for windows PE malware detection," *Electronics*, vol. 10, no. 4, p. 485, 2021.
- [7] N. A. Azeez, O. E. Odufuwa, S. Misra, J. Oluranti, and R. Damaševičius, "Windows PE malware detection using ensemble learning," *Informatics*, vol. 8, no. 1, p. 10, 2021.
- [8] H. Alamro, W. Mtouaa, S. Aljameel, A. S. Salama, M. A. Hamza, and A. Y. Othman, "Automated android malware detection using optimal ensemble learning approach for cybersecurity," *IEEE Access*, vol. 11, pp. 72509–72517, 2023.
- [9] K. S. Roy, T. Ahmed, P. B. Udas, M. E. Karim, and S. Majumdar, "MalHyStack: A hybrid stacked ensemble learning framework with feature engineering schemes for obfuscated malware analysis," *Intell. Syst. Appl.*, vol. 20, 2023, Art. no. 200283.
- [10] M. A. Hossain et al., "AI-enabled approach for enhancing obfuscated malware detection: A hybrid ensemble learning with combined feature selection techniques," *Int. J. Syst. Assur. Eng. Manage.*, pp. 1, 2024.
- [11] H. Juan Zhu, Y. Li, L. m Wang, and V. S. Sheng, "A multi-model ensemble learning framework for imbalanced android malware detection," *Expert Syst. Appl.*, vol. 234, 2023, Art. no. 120952.
- [12] X. Wang, L. Zhang, K. Zhao, X. Ding, and M. Yu, "MFDroid: A stacking ensemble learning framework for android malware detection," *Sensors*, vol. 22, no. 7, p. 2597, 2022.
- [13] S. H. Khan et al., "A new deep boosted CNN and ensemble learning based IoT malware detection," *Comput. Secur.*, vol. 133, 2023, Art. no. 103385.
- [14] D. Trizna, "Quo vadis: Hybrid machine learning meta-model based on contextual and behavioral malware representations," in *Proc. 15th ACM Workshop Artif. Intell. Security, AISec*, New York, NY, USA: ACM, 2022, pp. 127–136.
- [15] J. A. Johnny, K. Asmitha, P. Vinod, G. Radhamani, K. R. Rehiman, and M. Conti, "Deep learning fusion for effective malware detection: Leveraging visual features," *Cluster Comput.*, vol. 28, no. 2, 2025, Art. no. 135.
- [16] P. Thakur, V. Kansal, and V. Rishiwal, "Hybrid deep learning approach based on LSTM and CNN for malware detection," *Wireless Pers. Commun.*, vol. 136, no. 3, pp. 1879–1901, 2024.
- [17] A. K. A. V. P. R. R. K. A. N. Raveendran, M. Conti, and Vol, P, "Android malware defense through a hybrid multi-modal approach," *J. Netw. Comput. Appl.*, vol. 233, 2025, Art. no. 104035.
- [18] C. Wang, L. Zhang, K. Zhao, X. Ding, and X. Wang, "AdvAndMal: Adversarial training for android malware detection and family classification," *Symmetry*, vol. 13, no. 6, 2021, Art. no. 1081.
- [19] S. Alahmed, Q. Alasad, M. M. Hammood, J.-S. Yuan, and M. Alawad, "Mitigation of black-box attacks on intrusion detection systems-based ML," *Computers*, vol. 11, no. 7, 2022, Art. no. 115.
- [20] P. Wang, H.-C. Lin, J.-H. Chen, W.-H. Lin, and H.-C. Li, "Improving cyber defense against ransomware: A generative adversarial networks-based adversarial training approach for long short-term memory network classifier," *Electronics*, vol. 14, no. 4, 2025, Art. no. 810.
- [21] S. Li, J. Wang, Y. Wang, G. Zhou, and Y. Zhao, "EIFDAA: Evaluation of an ids with function-discarding adversarial attacks in the IIoT," *Heliyon*, vol. 9, no. 2, 2023.
- [22] I. Debicha, T. Debatty, J.-M. Dricot, and W. Mees, "Adversarial training for deep learning-based intrusion detection systems," 2021, *arXiv:2104.09852*.
- [23] F. De Gaspari, D. Hitaj, G. Pagnotta, L. De Carli, and L. V. Mancini, "Evading behavioral classifiers: A comprehensive analysis on evading ransomware detection techniques," *Neural Comput. Appl.*, vol. 34, no. 14, pp. 12077–12096, 2022.
- [24] L. Mauri and E. Damiani, "Hardening behavioral classifiers against polymorphic malware: An ensemble approach based on minority report," *Inf. Sci.*, vol. 689, 2025, Art. no. 121499.
- [25] A. Redhu, P. Choudhary, K. Srinivasan, and T. K. Das, "Deep learning-powered malware detection in cyberspace: A contemporary review," *Front. Phys.*, vol. 12, 2024, Art. no. 1349463.
- [26] H. S. Anderson and P. Roth, "Ember: An open dataset for training static pe malware machine learning models," 2018, *arXiv:1804.04637*.
- [27] N. Carlini et al., "On evaluating adversarial robustness," 2019, *arXiv:1902.06705*.
- [28] K. Aryal, M. Gupta, M. Abdelsalam, P. Kunwar, and B. Thuraishingham, "A survey on adversarial attacks for malware analysis," *IEEE Access*, vol. 13, pp. 428–459, 2025.
- [29] D. Marinho, "Study of common sub-graphs of system call dependency graphs for malware classification," Ph.D. dissertation, Inria Rennes-Bretagne Atlantique, Rennes, France, 2018.
- [30] C. E. Shannon, "A mathematical theory of communication," *Bell System Tech. J.*, vol. 27, no. 3, pp. 379–423, 1948.
- [31] L. Mauri, B. Apolloni, and E. Damiani, "Robust ML model ensembles via risk-driven anti-clustering of training data," *Inf. Sci.*, vol. 633, pp. 122–140, 2023.
- [32] A. Vassilev, A. Oprea, A. Fordyce, and H. Andersen, "Adversarial machine learning: A taxonomy and terminology of attacks and mitigations," National Institute of Standards and Technology, Gaithersburg, MD, USA, Tech. Rep., 2024.

- [33] S. Ennaji, F. De Gaspari, D. Hitaj, A. Kbidi, and L. V. Mancini, "Adversarial challenges in network intrusion detection systems: Research insights and future prospects," 2024, *arXiv:2409.18736*.
- [34] C. Liu, M. Salzmann, T. Lin, R. Tomioka, and S. Süssstrunk, "On the loss landscape of adversarial training: Identifying challenges and how to overcome them," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 21476–21487, 2020.
- [35] B. Vivek, K. R. Mopuri, and R. V. Babu, "Gray-box adversarial training," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 203–218.
- [36] G. Creech and J. Hu, "A semantic approach to host-based intrusion detection systems using contiguous and discontinuous system call patterns," *IEEE Trans. Comput.*, vol. 63, no. 4, pp. 807–819, Apr. 2014.
- [37] G. Karat, J. M. Kannimoola, N. Nair, A. Vazhayil, S. V. G., and P. Poornachandran, "CNN-LSTM hybrid model for enhanced malware analysis and detection," *Procedia Comput. Sci.*, vol. 233, pp. 492–503, 2024.
- [38] J. Sun, X. Luo, W. Wang, Y. Gao, and W. Zhao, "Robust malware identification via deep temporal convolutional network with symmetric cross entropy learning," *IET Softw.*, vol. 17, no. 4, pp. 392–404, 2023.
- [39] X. Wu, Y. Song, X. Hou, Z. Ma, and C. Chen, "Deep learning model with sequential features for malware classification," *Appl. Sci.*, vol. 12, no. 19, 2022, Art. no. 9994.
- [40] M. D. Preda, M. Christodorescu, S. Jha, and S. Debray, "A semantics-based approach to malware detection," *ACM SIGPLAN Notices*, vol. 42, no. 1, pp. 377–388, 2007.
- [41] V. Zuo, "A labelled version of the ADFA-LD dataset." [Online]. Available: <https://github.com/verazuo/a-labelled-version-of-the-adfa-lb-dataset/blob/master/adfa-lb2018>
- [42] G. Gebrehans et al., "Generative adversarial networks for dynamic malware behavior: A comprehensive review, categorization, and analysis," *IEEE Trans. Artif. Intell.*, vol. 6, no. 8, pp. 1955–1976, Aug. 2025.

Open Access funding provided by 'Universita degli Studi di Milano' within the CRUI CARE Agreement